ЛАБОРАТОРНА РОБОТА № 3

Тема: Одновимірні та багатовимірні масиви, обробка символьних рядків у мові С та робота з гілками в Git. Посилання на GitHub: https://github.com/VladyslavChepel/programming-sem1

Мета роботи: Набути практичних навичок обробки одновимірних масивів, кеш-дружньої обробки двовимірних матриць за принципом Row-Major Order та ручної обробки символьних рядків із нуль-термінатором у мові С, а також закріпити навички роботи з гілками в системі контролю версій Git.

Етап 0. Робота з гілками в Git

Спочатку в репозиторії programming-sem1 я створив гілку lab03 командою git checkout -b lab03 і каталог lab03 для файлів роботи. Кожне завдання фіксував окремим комітом: task1.c, task2.c, task3.c і наприкінці Report_Lab03.pdf. Потім перемкнувся на main (git checkout main), злив гілку командою git merge lab03 і відправив на GitHub обидві гілки: git push origin main та git push origin lab03.

Етап 1. Обробка одновимірного масиву

Мій номер у списку 30, тому варіант дорівнює (30 − 1) mod 15 + 1 = 15. Завдання: знайти довжину найдовшої неперервної послідовності однакових чисел, що йдуть підряд. Розмір масиву заданий константою #define SIZE 15. Масив заповнюється випадковими цілими числами з діапазону [−50; 50] за формулою rand() % 101 - 50, а перед цим викликається srand(time(NULL)), щоб числа відрізнялися від запуску до запуску. Для пошуку вистачає одного проходу циклом for і двох лічильників: curLen рахує довжину поточної серії, maxLen зберігає найбільшу знайдену. Обидва починаються з 1, бо один елемент уже є серією довжиною 1. Якщо a[i] дорівнює a[i-1], curLen збільшується, інакше повертається до 1. Коли curLen стає більшим за maxLen, оновлюється maxLen і запам’ятовується індекс початку серії. Алгоритм працює за O(n) і не потребує додаткового масиву.

Лістинг програми task1.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 15

int main(void)
{
    int a[SIZE];

    srand((unsigned)time(NULL));

    /* заповнення випадковими числами з [-50; 50] */
    for (int i = 0; i < SIZE; i++) {
        a[i] = rand() % 101 - 50;
    }

    printf("Масив: ");
    for (int i = 0; i < SIZE; i++) {
        printf("%4d", a[i]);
    }
    printf("\n");

    /* пошук найдовшої неперервної послідовності однакових чисел */
    int maxLen = 1;     /* найкраща довжина */
    int curLen = 1;     /* довжина поточної серії */
    int maxStart = 0;   /* індекс початку найкращої серії */

    for (int i = 1; i < SIZE; i++) {
        if (a[i] == a[i - 1]) {
            curLen++;
        } else {
            curLen = 1;
        }
        if (curLen > maxLen) {
            maxLen = curLen;
            maxStart = i - curLen + 1;
        }
    }

    printf("Довжина найдовшої послідовності однакових чисел: %d\n", maxLen);
    printf("Значення: %d, починається з індексу %d\n", a[maxStart], maxStart);

    return 0;
}
```

Компіляція виконувалася командою: gcc -Wall -Wextra task1.c -o task1

Результати тестування:

У першому запуску в масиві є два однакові сусідні числа (−11 і −11), тому найдовша послідовність має довжину 2:

```
Масив:   46  18  38 -17  44  37  48 -18  29 -11 -11  10  49 -43  31
Довжина найдовшої послідовності однакових чисел: 2
Значення: -11, починається з індексу 9
```

У другому запуску однакових сусідів немає, і програма вивела довжину 1:

```
Масив:  -39  21 -41 -22  -5  19  26  30  12 -21  20 -37  35  -2  12
Довжина найдовшої послідовності однакових чисел: 1
Значення: -39, починається з індексу 0
```

Етап 2. Кеш-дружня обробка 2D-матриць

У другому завданні квадратну матрицю 4 × 4 треба перетворити на верхньотрикутну, тобто замінити нулями всі елементи строго під головною діагоналлю (де i > j). Розмір задано константою #define N 4, а матриця заповнюється випадковими числами з діапазону [−9; 9]. Усі вкладені цикли написані за принципом Row-Major Order: зовнішній цикл іде по рядках (i), внутрішній по стовпчиках (j). Так програма читає елементи в тому самому порядку, в якому вони лежать у пам’яті, і кеш процесора працює ефективно. Матриця виводиться функцією printMatrix з форматом %6d, тому стовпчики вирівняні, і до та після обробки її легко порівняти.

Лістинг програми task2.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 4

static void printMatrix(int m[N][N])
{
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%6d", m[i][j]);
        }
        printf("\n");
    }
}

int main(void)
{
    int m[N][N];

    srand((unsigned)time(NULL));

    /* Row-Major: зовнішній цикл - рядки, внутрішній - стовпчики */
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            m[i][j] = rand() % 19 - 9;
        }
    }

    printf("Матриця до обробки:\n");
    printMatrix(m);

    /* верхньотрикутна форма: усе строго під головною діагоналлю (i > j) = 0 */
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (i > j) {
                m[i][j] = 0;
            }
        }
    }

    printf("\nМатриця після обробки (верхньотрикутна):\n");
    printMatrix(m);

    return 0;
}
```

Компіляція виконувалася командою: gcc -Wall -Wextra task2.c -o task2

Результати тестування:

Після обробки всі елементи під головною діагоналлю стали нулями, а діагональ і верхня частина матриці залишилися без змін.

```
Матриця до обробки:
    -2     6    -1     5
     9     1     6     7
    -5     9     5     2
     1    -5    -7    -1

Матриця після обробки (верхньотрикутна):
    -2     6    -1     5
     0     1     6     7
     0     0     5     2
     0     0     0    -1
```

Етап 3. Ручна низькорівнева обробка С-рядків

У третьому завданні потрібно окремо порахувати літери, цифри та розділові знаки у введеному тексті. Буфер оголошений як char str[MAX_LEN + 1]: зайвий байт потрібен під нуль-термінатор \0. Рядок читається через fgets(str, sizeof(str), stdin), бо ця функція не дозволяє вийти за межі буфера. Аналіз я зробив вручну, циклом for (i = 0; str[i] != '\0'; i++), без функцій з <string.h>. Кожен символ перевіряється за кодами ASCII. Літери це A–Z і a–z, цифри це 0–9, а розділовими знаками вважаються решта друкованих символів, крім пробілу (коди 33–47, 58–64, 91–96 і 123–126). Пробіли та символ \n, який fgets залишає в кінці рядка, не потрапляють у жодну групу. Сам рядок програма не змінює, тому нуль-термінатор залишається на своєму місці.

Лістинг програми task3.c

```c
#include <stdio.h>

#define MAX_LEN 100

int main(void)
{
    char str[MAX_LEN + 1];   /* правило N + 1: місце під '\0' */
    int letters = 0, digits = 0, punct = 0;

    printf("Введіть текст: ");
    if (fgets(str, sizeof(str), stdin) == NULL) {
        return 1;
    }

    /* покрокове сканування до нуль-термінатора */
    for (int i = 0; str[i] != '\0'; i++) {
        char c = str[i];

        if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')) {
            letters++;
        } else if (c >= '0' && c <= '9') {
            digits++;
        } else if ((c >= 33 && c <= 47) || (c >= 58 && c <= 64) ||
                   (c >= 91 && c <= 96) || (c >= 123 && c <= 126)) {
            punct++;   /* символи пунктуації з таблиці ASCII (без пробілу) */
        }
    }

    printf("Літер: %d\nЦифр: %d\nРозділових знаків: %d\n", letters, digits, punct);

    return 0;
}
```

Компіляція виконувалася командою: gcc -Wall -Wextra task3.c -o task3

Результати тестування:

Для перевірки я ввів два рядки. У першому 20 літер, 5 цифр (3, 2, 0, 2, 6) і 6 розділових знаків (кома, знак оклику, двокрапка, дефіс і дві дужки):

```
Введіть текст: Hello, World! Lab 3: gcc -Wall (2026)
Літер: 20
Цифр: 5
Розділових знаків: 6
```

У другому 20 літер, 3 цифри (1, 0, 0) і 7 розділових знаків (три крапки, відсоток і три знаки оклику):

```
Введіть текст: Programming is fun... 100% sure!!!
Літер: 20
Цифр: 3
Розділових знаків: 7
```

Висновок

У цій лабораторній роботі я написав три програми мовою C і попрацював з гілками в Git. У першому завданні знайшов довжину найдовшої серії однакових чисел в масиві за один прохід з двома лічильниками. У другому перетворив квадратну матрицю на верхньотрикутну, а цикли організував по рядках, як вимагає Row-Major Order: пам’ять читається послідовно, і кеш використовується краще. У третьому порахував літери, цифри та розділові знаки вручну, скануючи масив char до нуль-термінатора. Для нього в буфері залишено один зайвий байт, а рядок читається через fgets. Результати всіх тестових запусків збіглися з ручним підрахунком, а компіляція з -Wall -Wextra пройшла без попереджень. Кожне завдання зафіксовано окремим комітом у гілці lab03, потім гілку злито в main та відправлено на GitHub.